# Information Security CSC232

LECTURE-6 14-OCT-2026

SAAD MALIK

# Lecture Contents

**Part 1: Access Control Fundamentals**
- Identification, Authentication, Authorization, and Accountability
- Subjects, Objects, Permissions, and Access Rights
- Access Control Lists (ACLs)
- Access Control in Organizational Systems

**Part 2: Authentication Methods**
- Authentication Factors
- Passwords, PINs, Tokens, and Biometrics
- Multi-Factor Authentication
- Password Security and Secure Storage

**Part 3: Access Control Models**
- Discretionary Access Control (DAC)
- Mandatory Access Control (MAC)
- Role-Based Access Control (RBAC)
- Attribute-Based Access Control (ABAC)

# What is Access Control?

- ITU-T Recommendation X.800 Definition:
"Access control is the prevention of unauthorized use of a resource, including the prevention of use of a resource in an unauthorized manner."

- RFC 2828 Definition:
"Measures that implement and assure security services in a computer system, particularly those that assure access control service."

- Key Objectives:
  - Prevent unauthorized users from gaining access to resources
  - Prevent legitimate users from accessing resources in an unauthorized manner
  - Enable legitimate users to access resources in an authorized manner

- Key Point: Access control is the central element of computer security.

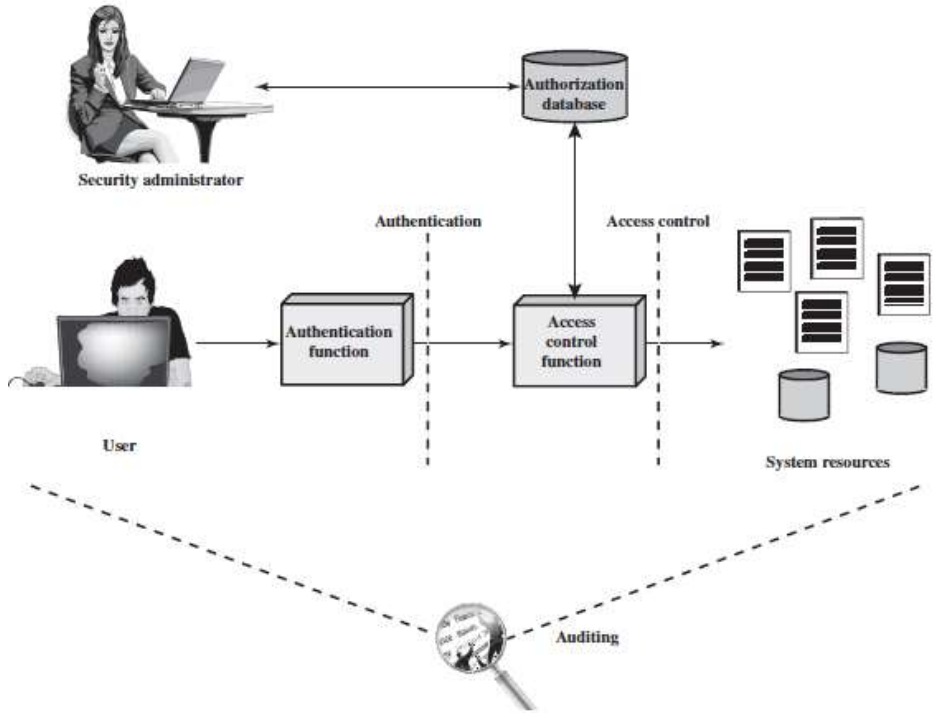



Figure 4.1 **Relationship among Access Control and Other Security Functions**
*Source:* Based on [SAND94]

# The AAA Framework — Identification, Authentication, Authorization, Accounting

| Function | Definition | Key Question |
|---|---|---|
| Identification | Mechanism whereby an unverified entity (supplicant) proposes a label by which they are known to the system | Who do you claim to be? |
| Authentication | Verification that credentials of a user or system entity are valid | Who are you really? |
| Authorization | Granting of a right or permission to access a system resource | What can you do? |
| Audit/Accounting | Independent review of system records and activities | What did you do? |

**How They Work Together:**

1. **Identification:** Supplicant presents an identifier (e.g., username)
2. **Authentication:** System verifies the credentials (e.g., password, OTP)
3. **Authorization:** Access control function determines if the specific requested access is permitted
4. **Accounting:** Auditing function monitors and records user accesses

**Real-World Example:**
When you log into your bank account:

- **Identification:** You enter your username (e.g., "ali.khan")
- **Authentication:** You enter password + OTP
- **Authorization:** You can view balance but cannot approve loans
- **Accounting:** Bank logs every transaction you make

# Access Control Policies — Three Categories

| Policy Type | Definition | Key Characteristic |
|---|---|---|
| Discretionary Access Control (DAC) | Controls access based on identity of requestor and access rules | Entity can enable another entity to access a resource by its own volition |
| Mandatory Access Control (MAC) | Controls access based on comparing security labels with security clearances | Entity with clearance cannot enable another entity to access a resource |
| Role-Based Access Control (RBAC) | Controls access based on roles users have within the system | Access rights assigned to roles, not individual users |
| Attribute Based Access Control (ABAC) | | |

**Key Point:** These three policies are **not mutually exclusive** (Figure). A system can employ two or even all three policies for different classes of resources.

**Real-World Example:**

- **DAC:** Google Drive — You can share your document with anyone
- **MAC:** Military classified systems — Only Top Secret clearance can access Top Secret documents
- **RBAC:** Hospital system — Doctors can view patient records; nurses can update vitals; admin staff can schedule appointments

# Access Control Requirements

| Requirement | Description | Example |
|---|---|---|
| Reliable Input | Authentication mechanism as front end; reliable source identification | Source IP address validation |
| Fine and Coarse Specifications | Support access regulation at individual record/field level; also coarse-grained for efficiency | Database row-level security vs. table-level access |
| Least Privilege | Each entity granted minimum resources and authorizations needed | Developer gets only required DB access, not admin rights |
| Separation of Duty | Divide steps in a system function among different individuals | Bank check requires preparer, signer, issuer |
| Open and Closed Policies | Closed: Only specifically authorized accesses allowed; Open: Only prohibited accesses specified | Closed = default deny; Open = default allow |
| Policy Combinations | Multiple policies may apply; conflict resolution needed | DAC + RBAC for different resource classes |
| Administrative Policies | Specify who can add, delete, or modify authorization rules | Security administrator role |
| Dual Control | Task requires two or more individuals working in tandem | Nuclear launch requires two keys |

# Subjects, Objects, and Access Rights

## Basic Elements of Access Control:

| Element | Definition | Examples |
|---|---|---|
| Subject | Entity capable of accessing objects; generally equates with process | User, application, process |
| Object | Resource to which access is controlled | Records, files, directories, mailboxes, programs |
| Access Right | Way in which a subject may access an object | Read, Write, Execute, Delete, Create, Search |

**Three Classes of Subject:**

1. **Owner:** Creator of a resource (or system administrator for system resources)
2. **Group:** Named group of users granted access rights; membership sufficient to exercise rights
3. **World/Others:** Least amount of access granted to users not in owner or group categories

**Real-World Example:**

- **Subject:** You (the user)
- **Object:** Your company's HR database
- **Access Rights:** You can Read your own record, but cannot Write or Delete it

# Access Rights — Detailed

| Access Right | Description | Includes |
|---|---|---|
| Read | User may view information in a system resource | Ability to copy or print |
| Write | User may add, modify, or delete data | Includes read access |
| Execute | User may execute specified programs | — |
| Delete | User may delete certain system resources | Files or records |
| Create | User may create new files, records, or fields | — |
| Search | User may list files in a directory or search the directory | — |

**Real-World Example — UNIX File Permissions:**

-rw-r--r-- 1 user group 1024 Jan 15 10:30 file.txt

- Owner: read + write
- Group: read only
- Others: read only

# Discretionary Access Control (DAC)

**Definition:**
Controls access based on the identity of the requestor and on access rules stating what requestors are (or are not) allowed to do.

**Key Characteristic:**
An entity might have access rights that permit the entity, by its own volition, to enable another entity to access some resource.

**Access Matrix (Figure 4.3a):**

| | File 1 | File 2 | File 3 | File 4 |
|---|---|---|---|---|
| User A | Own, R, W | — | Own, R, W | — |
| User B | R | Own, R, W, C | W | R |
| User C | R | R | — | Own, R, W |

**Decomposition:**

- **By Columns:** Access Control Lists (ACLs)
- **By Rows:** Capability Tickets

**Real-World Example:**
Google Drive — You own a document and can share it with anyone you choose. That person can then share it with others (if you allow).

# Access Control Matrix & Access Control Lists (ACLs)

**Definition:**
An ACL is a decomposition of the access matrix by columns. For each object, an ACL lists users and their permitted access rights.

**Access Matrix Example (Figure 4.3a):**

| | File 1 | File 2 | File 3 | File 4 |
|---|---|---|---|---|
| User A | Own, R, W | — | Own, R, W | — |
| User B | R | Own, R, W, C | W | R |
| User C | R | R | — | Own, R, W |

**ACL for File 1 (Figure 4.3b):**

- User A: Own, Read, Write
- User B: Read
- User C: Read

**ACL for File 2:**

- User B: Own, Read, Write
- User C: Read

**Key Point:** ACLs are convenient for determining who has access to a **particular resource**, but not for determining all access rights of a **specific user**.

**Real-World Example:**
Windows File Explorer → Right-click file → Properties → Security tab shows the ACL (who can read, write, execute).

# Capability Tickets vs. ACLs

| Feature | Access Control Lists (ACLs) | Capability Tickets |
|---|---|---|
| Decomposition | By columns (objects) | By rows (subjects) |
| Convenient For | Determining who has access to a resource | Determining what access a user has |
| Security Concern | Less problematic | Greater security problem — tickets may be dispersed |
| Integrity | Protected by OS | Ticket must be unforgeable (OS holds all tickets or cryptographic token) |
| Real-World Example | Windows file permissions | Kerberos tickets |

**Authorization Table (Table 4.1):**
A data structure that contains one row for one access right of one subject to one resource.

- Sorting by subject = Capability list
- Sorting by object = ACL

# Access Control in Organizational Systems

| Component | Role | Example |
|---|---|---|
| Operating System | Native access control component | Windows ACLs, UNIX permissions |
| Add-on Security Packages | Supplement native OS capabilities | SELinux, AppArmor |
| Applications | Incorporate access control functions | Database management systems |
| External Devices | Provide access control services | Firewalls, routers |

**How Access Control is Implemented:**

**Real-World Example — Corporate Environment:**

- **OS Level:** Windows Server with Active Directory
- **Application Level:** SAP ERP with role-based access
- **Network Level:** Cisco Firewall with ACLs
- **Physical Level:** Badge access to server room

**Key Point:** Access control is a **layered defense** — multiple components work together.

# Extended Access Control Model

**Protection State:** The set of information, at a given point in time, that specifies the access rights for each subject with respect to each object (can be a process, device, memory location or subject).

**Three Requirements:**

1. Representing the protection state
2. Enforcing access rights
3. Allowing subjects to alter the protection state in certain ways

**Extended Access Control Matrix (Figure 4.4):**

| | | OBJECTS | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | Subjects | | | Files | | Processes | | Disk drives | |
| | | $S_1$ | $S_2$ | $S_3$ | $F_1$ | $F_2$ | $P_1$ | $P_2$ | $D_1$ | $D_2$ |
| SUBJECTS | $S_1$ | control | owner | owner control | read* | read owner | wakeup | wakeup | seek | owner |
| | $S_2$ | | control | | write* | execute | | | owner | seek* |
| | $S_3$ | | | control | | write | stop | | | |

* = copy flag set

Figure 4.4 **Extended Access Control Matrix**

**Access Attributes:** Strings that specify access rights of subject S to object X.

# Protection Domains

**Definition:**
A protection domain is a set of objects together with access rights to those objects.

**Key Concepts:**

- In the access matrix, a **row** defines a protection domain
- Each user has a protection domain
- Processes spawned by the user have access rights defined by the same protection domain
- A user can spawn processes with a **subset** of their access rights (new protection domain)

**User Mode vs. Kernel Mode:**

| Mode | Description | Access |
|---|---|---|
| User Mode | User program executes | Certain areas of memory protected; certain instructions not executable |
| Kernel Mode | System routine executes | Privileged instructions may execute; protected memory accessible |

**Real-World Example:**
When you run a web browser (user mode), it cannot directly access hardware. When it needs to read a file, it makes a system call, and the OS (kernel mode) performs the operation.

# UNIX File Access Control

**Traditional UNIX File Access Control:**

Each file has:

- **Owner ID:** User who created the file
- **Group ID:** Group to which the file belongs
- **12 Protection Bits:**
- 9 bits: read, write, execute for owner, group, others
- 3 bits: SetUID, SetGID, Sticky

**Example:**

-rw-r--r-- 1 user group 1024 Jan 15 10:30 file.txt

- Owner: read + write
- Group: read only
- Others: read only

**Special Permissions:**

| Permission | Description | Effect |
|---|---|---|
| SetUID | Set user ID | When executed, process runs with owner's rights |
| SetGID | Set group ID | When executed, process runs with group's rights; for directories, new files inherit group |
| Sticky | Sticky bit | For directories, only owner can rename, move, or delete files |

**Superuser:** User ID designated as "superuser" is exempt from usual file access control constraints.

# UNIX Access Control Lists (ACLs)

**Extended ACLs (Figure 4.6b):**

- Many modern UNIX systems support ACLs (FreeBSD, OpenBSD, Linux, Solaris)
- Traditional UNIX approach = **minimal ACL**
- Extended ACL = **extended access control list**

**FreeBSD Strategy:**

1. Owner class and other class entries have same meaning as minimal ACL
2. Group class entry specifies permissions for owner group; functions as a **mask**
3. Additional named users and named groups may be associated with the file

**Example:**

user::rw-
user:joe:rw-
group::r--
mask::rw-
other::---

**How Access is Determined:**

1. Select ACL entry that most closely matches requesting process (owner → named users → groups → others)
2. Check if matching entry contains sufficient permissions

**Real-World Example:**

Linux setfacl command allows administrators to assign ACLs to files, providing more flexible access control than traditional UNIX permissions.

# Role-Based Access Control (RBAC)

**Definition:**
Controls access based on the roles that users have within the system and on rules stating what accesses are allowed to users in given roles.

**Key Concepts:**

- Access rights assigned to **roles**, not individual users
- Users assigned to roles based on responsibilities
- Many-to-many relationship between users and roles
- Many-to-many relationship between roles and permissions

**Access Control Matrix Representation (Figure 4.8):**

**Upper Matrix (Users to Roles):**

**Lower Matrix (Roles to Objects):**

**Real-World Example:**
**Dresdner Bank RBAC System (Section 4.6):**

- 65 official positions × 368 job functions = 23,920 potential roles
- Only 1,300 roles in current use
- 42,000 security profiles distributed daily
- Three administrative units for greater security

| | | OBJECTS | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| | | $R_1$ | $R_2$ | $R_n$ | $F_1$ | $F_2$ | $P_1$ | $P_2$ | $D_1$ | $D_2$ |
| ROLES | $R_1$ | control | owner | owner control | read * | read owner | wakeup | wakeup | seek | owner |
| | $R_2$ | | control | | write * | execute | | | owner | seek * |
| | • | | | | | | | | | |
| | • | | | | | | | | | |
| | • | | | | | | | | | |
| | $R_n$ | | | control | | write | stop | | | |

Figure 4.8 **Access Control Matrix Representation of RBAC**

# RBAC Reference Models

**Family of RBAC Models (Figure 4.9):**

| Model | Hierarchies | Constraints | Description |
|---|---|---|---|
| RBAC0 | No | No | Base model — minimum functionality |
| RBAC1 | Yes | No | Adds role hierarchies |
| RBAC2 | No | Yes | Adds constraints |
| RBAC3 | Yes | Yes | Consolidated model — includes RBAC1 and RBAC2 |

**RBAC0 Entities:**

- **User:** Individual with user ID
- **Role:** Named job function
- **Permission:** Approval of particular mode of access
- **Session:** Mapping between user and activated subset of roles

**Role Hierarchies (RBAC1) — Figure 4.10:**

- Senior roles inherit permissions from junior roles
- Example: Project Lead inherits from Production Engineer and Quality Engineer
- Production Engineer and Quality Engineer both inherit from Engineer

**Constraints (RBAC2):**

- **Mutually Exclusive Roles:** User can be assigned to only one role in set
- **Cardinality:** Maximum number of users that can be assigned to a role
- **Prerequisite Roles:** User can only be assigned to a role if already assigned to another role

# NIST RBAC Model

**Four Model Components (Figure 4.11):**

**Core RBAC Definitions:**

- **Object:** Any system resource subject to access control
- **Operation:** Executable image of a program
- **Permission:** Approval to perform an operation on one or more objects

**Administrative Functions:**

- Add/delete users and roles
- Create/delete user-to-role assignments
- Create/delete permission-to-role assignments

**Supporting System Functions:**

- Create user session
- Add/delete active role to session
- Check if session subject has permission

**Review Functions:**

- View all elements and relations (users, roles, assignments, sessions)

| Component | Description |
|---|---|
| Core RBAC | Users, roles, permissions, sessions |
| Hierarchical RBAC | Role hierarchies (general and limited) |
| Static Separation of Duty (SSD) | Mutually exclusive roles; cardinality constraints |
| Dynamic Separation of Duty (DSD) | Constraints on roles activated within or across sessions |

# Attribute-Based Access Control (ABAC)

**Definition:**
Controls access based on attributes of the subject, object, and environment.

**Key Concepts:**

- **Subject Attributes:** User's identity, role, department, clearance level
- **Object Attributes:** File's classification, owner, creation date
- **Environmental Attributes:** Time of day, location, threat level
- **Policy:** Rules that combine attributes to make access decisions

**Comparison with Other Models:**

| Model | Basis for Access Decision |
|---|---|
| DAC | Identity of requestor |
| MAC | Security labels and clearances |
| RBAC | Roles users have within the system |
| ABAC | Attributes of subject, object, and environment |

**Real-World Example:**
A hospital system where access to patient records is determined by:

- **Subject Attributes:** Doctor's department, patient assignment
- **Object Attributes:** Record sensitivity, patient consent status
- **Environmental Attributes:** Time of day, emergency status

**Key Point:** ABAC provides the most **fine-grained** access control but is also the most **complex** to manage.

# Access Control Model Comparison

| Feature | DAC | MAC | RBAC | ABAC |
|---|---|---|---|---|
| Basis | Identity | Security labels | Roles | Attributes |
| Flexibility | High | Low | Medium | Very High |
| Administrative Burden | High | Low | Medium | High |
| Granularity | Coarse to Fine | Coarse | Medium | Very Fine |
| Real-World Use | File sharing | Military | Enterprise | Cloud, IoT |
| Example | Google Drive | Classified systems | Bank systems | AWS IAM |

**Key Point:** These models are **not mutually exclusive**. A system can employ multiple policies for different classes of resources.

# What is User Authentication?

**RFC 2828 Definition:**
*"The process of verifying an identity claimed by or for a system entity."*

**Two Steps:**
1. **Identification Step:** Presenting an identifier to the security system
2. **Verification Step:** Presenting or generating authentication information that corroborates the binding between the entity and the identifier

**Key Distinction:**
- **User Authentication:** Verifying identity of a user
- **Message Authentication:** Verifying contents of a message have not been altered and source is authentic

**Key Point:** User authentication is the **fundamental building block** and **primary line of defense** in most computer security contexts.

# Four Means of Authentication

| Means | Description | Examples |
|---|---|---|
| Something the individual knows | Information known only to the user | Password, PIN, answers to prearranged questions |
| Something the individual possesses | Physical object the user has | Electronic key cards, smart cards, physical keys (token) |
| Something the individual is (static biometrics) | Physical characteristic | Fingerprint, retina, face |
| Something the individual does (dynamic biometrics) | Behavioral characteristic | Voice pattern, handwriting, typing rhythm |

**Key Point:** All methods, properly implemented, can provide secure authentication. Each has problems:

- Passwords can be guessed or stolen
- Tokens can be forged or stolen
- Biometrics have false positives/negatives, cost, and user acceptance issues

**Real-World Example — Multi-Factor Authentication (MFA):**

- **Know:** Password
- **Have:** Phone (OTP)
- **Are:** Fingerprint

# Password-Based Authentication

**How It Works:**

1. User provides ID and password
2. System compares password to previously stored password for that user ID
3. Password file maintained by the system

**The ID Provides Security In Three Ways:**

1. Determines whether user is authorized to gain access
2. Determines privileges accorded to the user (superuser, guest, etc.)
3. Used in discretionary access control (granting file permissions)

**Real-World Example:**

Every time you log into email, social media, or online banking, you use password-based authentication.

# Password Vulnerabilities

| Attack Type | Description | Countermeasure |
|---|---|---|
| Offline Dictionary Attack | Attacker obtains password file, compares hashes against common passwords | Controls to prevent unauthorized access; intrusion detection; rapid reissuance |
| Specific Account Attack | Attacker targets specific account, submits guesses until correct | Account lockout after 3-5 failed attempts |
| Popular Password Attack | Uses popular password against wide range of user IDs | Policies to inhibit common passwords; scan IP addresses for patterns |
| Password Guessing Against Single User | Attacker gains knowledge about user, guesses password | Training and enforcement of password policies |
| Workstation Hijacking | Attacker waits until logged-in workstation unattended | Automatic logout after inactivity; intrusion detection |
| Exploiting User Mistakes | Users write down passwords, share them, or use preconfigured passwords | User training; intrusion detection; simpler passwords + another mechanism |
| Exploiting Multiple Password Use | Same password on multiple devices | Policy forbidding same/similar password on different devices |
| Electronic Monitoring | Password communicated across network is vulnerable to eavesdropping | Encryption (but encrypted password can be reused) |

# Hashed Passwords and Salt

**How UNIX Password Scheme Works (Figure 3.1):**

**Loading a New Password:**

1. User selects password
2. Password combined with **salt value** (random number)
3. Password + salt → hash function → fixed-length hash code
4. Hashed password + plaintext salt stored in password file

**Verifying a Password:**

1. User provides ID and password
2. OS uses ID to retrieve salt and encrypted password
3. Salt + user-supplied password → hash function
4. If result matches stored value, password accepted

**Three Purposes of Salt:**

1. Prevents duplicate passwords from being visible
2. Greatly increases difficulty of offline dictionary attacks (2^b factor)
3. Nearly impossible to determine if same password used on multiple systems

**Real-World Example:**

Linux /etc/shadow file stores hashed passwords with salt. Even if stolen, attacker must crack each password individually.

# Password Cracking Approaches

| Approach | Description | Example |
|---|---|---|
| Dictionary Attack | Try large dictionary of possible passwords | Try "password", "123456", "admin" |
| Rainbow Table | Precomputed hash values for each possible salt | 1.4 GB data cracks 99.9% of Windows passwords in 13.8 seconds |
| Brute Force | Try all possible combinations | Not feasible for long passwords |

**Observed Password Lengths (Table 3.1):**

- 3 characters or fewer: ~3% of passwords
- 6 characters or fewer: ~14% of passwords
- 8 characters: 42% of passwords

**Passwords Cracked from 13,797 Accounts (Table 3.2):**

- Total matched: 3,340 (24.2%)
- System dictionary: 1,027 (7.4%)
- Common names: 548 (4.0%)
- User/account name: 368 (2.7%)

**Key Insight:** Nearly one-fourth of passwords were guessed using a comprehensive dictionary and permutation strategy.

# Password Selection Strategies

| Strategy | Description | Pros | Cons |
|---|---|---|---|
| User Education | Tell users importance of hard-to-guess passwords | Simple | Often ignored; users poor judges |
| Computer-Generated Passwords | System generates random passwords | Very secure | Users cannot remember; write them down |
| Reactive Password Checking | System runs password cracker periodically | Finds weak passwords | Resource intensive; existing passwords vulnerable until found |
| Proactive Password Checking | System checks password at selection time | Balances security and usability | Must balance user acceptability and strength |

**Good Technique for Choosing Passwords:**
Use the first letter of each word of a phrase.

- *"My dog's first name is Rex"* → **MdfniR**
- *"My sister Peg is 24 years old"* → **MsPi24yo**

**Key Point:** Studies show users can remember such passwords but they are not susceptible to guessing attacks.

# Bloom Filter for Password Checking

**How It Works:**

1. Define k independent hash functions
2. Create hash table of N bits, all initially 0
3. For each password in dictionary, calculate k hash values; set corresponding bits to 1
4. When new password presented, calculate k hash values
5. If all corresponding bits are 1, reject password (it's in dictionary)

**False Positives:** Some passwords not in dictionary may be rejected (all bits happen to be 1)

**Probability of False Positive:**

$P \approx (1 - e^{(-kD/N)})^k$

Where:

- k = number of hash functions
- N = number of bits in hash table
- D = number of words in dictionary

**Performance (Figure 3.2):**

- Dictionary of 1 million words
- 0.01 probability of false positive
- 6 hash functions → need $9.6 \times 10^6$ bits (1.2 MB)
- Full dictionary storage: ~8 MB
- **Compression factor: ~7x**

**Real-World Example:**

Linux uses Bloom filter-based proactive password checking (pam_passwdqc).

# Token-Based Authentication

**Definition:**
Objects that a user possesses for the purpose of user authentication are called **tokens**.

**Types of Cards Used as Tokens (Table 3.3):**

| Card Type | Defining Feature | Example |
|---|---|---|
| Embossed | Raised characters only, on front | Old credit card |
| Magnetic Stripe | Magnetic bar on back, characters on front | Bank card |
| Memory | Electronic memory inside | Prepaid phone card |
| Smart | Electronic memory and processor inside | Biometric ID card |
| Contact | Electrical contacts exposed on surface | Smart card with reader |
| Contactless | Radio antenna embedded inside | NFC-enabled card |

**Memory Cards:**

- Can store but not process data
- Most common: bank card with magnetic stripe
- Used with password or PIN for computer authentication
- **Example:** ATM card + PIN

**Smart Cards:**

- Contains entire microprocessor (processor, memory, I/O ports)
- Three types of memory: ROM, EEPROM, RAM
- Can be contact or contactless
- **Example:** Biometric ID card

# Smart Card Authentication Protocols

| Protocol Type | Description | Example |
|---|---|---|
| Static | User authenticates to token; token authenticates to computer | Similar to memory token operation |
| Dynamic Password Generator | Token generates unique password periodically (e.g., every minute) | RSA SecurID |
| Challenge-Response | Computer generates challenge; token generates response | Public-key cryptography; token encrypts challenge with private key |

**Smart Card/Reader Exchange (Figure 3.4):**

1. Card inserted into reader → reset initiated
2. Card responds with **Answer to Reset (ATR)** message
3. Protocol negotiation (PTS command)
4. Command APDU / Response APDU exchange

**Real-World Example:**
**RSA SecurID Breach (2011):** Spear phishing emails with Excel attachment compromised SecurID tokens. Cost: $66 million. Lesson: Even security companies can be socially engineered.

# Biometric Authentication

**Definition:**
A biometric authentication system attempts to authenticate an individual based on unique physical characteristics.

**Physical Characteristics Used (Figure 3.5 — Cost vs. Accuracy):**

**Key Point:** Iris biometric systems have virtually zero false match rate.

| Characteristic | Description | Accuracy | Cost |
|---|---|---|---|
| Facial Characteristics | Relative location and shape of key facial features | Low | Low |
| Fingerprints | Pattern of ridges and furrows on fingertip | Medium | Medium |
| Hand Geometry | Shape, lengths, and widths of fingers | Medium | Medium |
| Retinal Pattern | Pattern formed by veins beneath retinal surface | High | High |
| Iris | Detailed structure of the iris | Very High | High |
| Signature | Unique style of handwriting | Low | Low |
| Voice | Physical and anatomical characteristics of speaker | Low | Low |

# Operation of a Biometric System

**Enrollment (Figure 3.6a):**

1. User presents name and password/PIN
2. System senses biometric characteristic
3. System digitizes input and extracts features (template)
4. User is enrolled — name, PIN, and biometric value stored

**Verification (Figure 3.6b):**

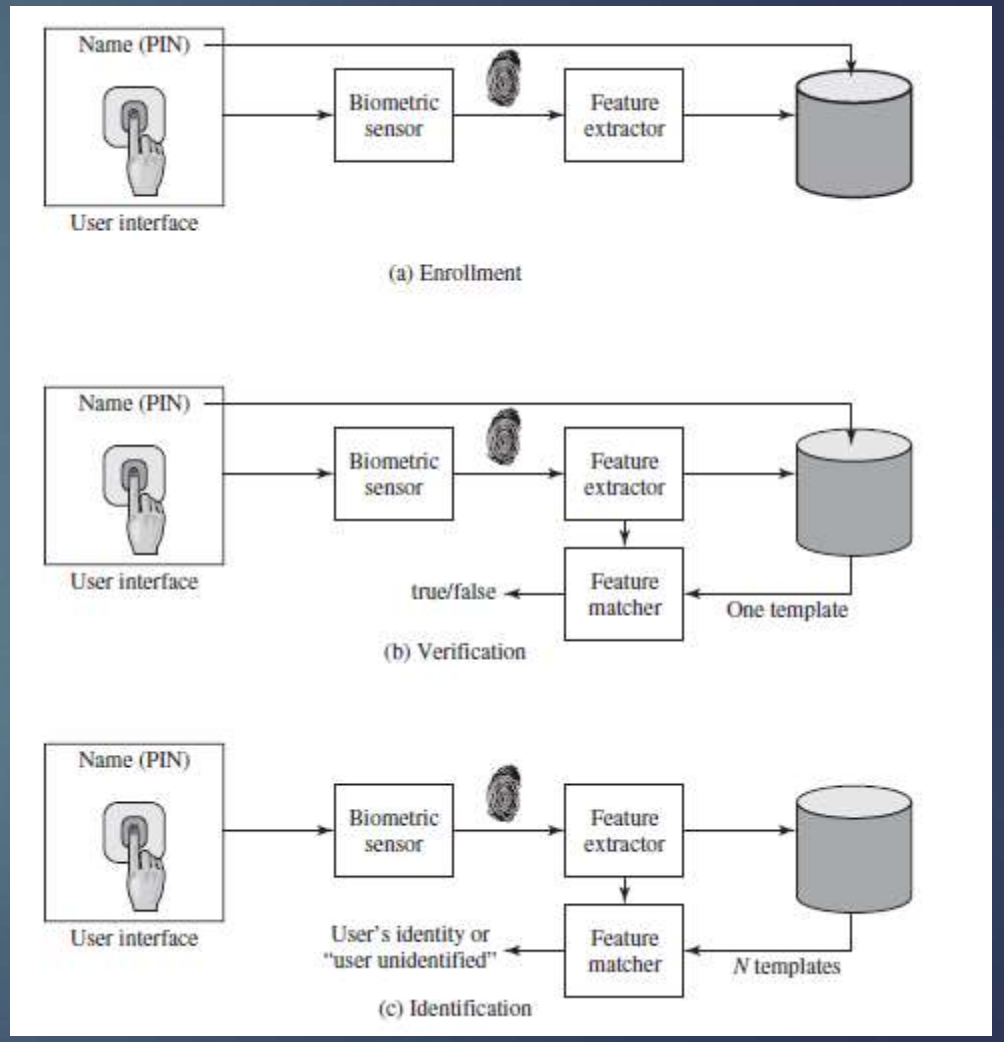
1. User enters PIN and uses biometric sensor
2. System extracts feature and compares to stored template
3. If match → user authenticated

**Identification (Figure 3.6c):**

1. User uses biometric sensor only (no additional info)
2. System compares presented template with all stored templates
3. If match → user identified; otherwise rejected

**Real-World Example — Iris Biometric System (Section 3.7):**

- Used by Nationwide Building Society in Swindon, England (1998)
- 91% preferred iris identification to PIN or signature
- 94% would recommend to friends and family
- **Epilogue:** System made up 25% of ATM cost; cancelled in 2003



# Biometric Accuracy

- Matching Score:
System uses algorithm to generate a matching score (typically a single number) that quantifies similarity between input and stored template.

The Dilemma (Figure 3.7):

- Genuine user: Matching score varies (bell curve)
- Imposter: Lower matching score (bell curve)
- Ranges overlap → threshold (t) must be selected
- False Match: s ≥ t when user is imposter
- False Nonmatch: s < t when user is genuine

Operating Characteristic Curve (Figure 3.8):

- System A: Consistently inferior to System B
- High-security applications: Low false match rate (point farther left)
- Forensic applications: Low false nonmatch rate (point farther right)
- Civilian applications: Equal error rates (middle)

Actual Biometric Measurement (Figure 3.9):

- Iris: No false matches in over 2 million cross-comparisons
- Face: Worst performer over broad range
- Fingerprint, Voice, Hand: Intermediate

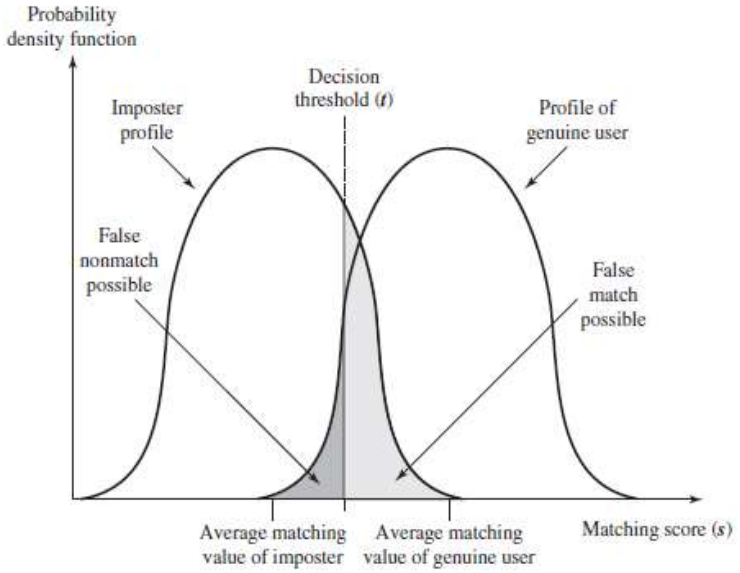



Figure 3.7 **Profiles of a Biometric Characteristic of an Imposter and an Authorized User** In this depiction, the comparison between the presented feature and a reference feature is reduced to a single numeric value. If the input value ($s$) is greater than a preassigned threshold ($t$), a match is declared.

# Remote User Authentication

| Protocol | How It Works | Defends Against |
|---|---|---|
| Password Protocol | Host sends nonce r; user returns f(r', h(P')) | Replay attacks; password theft |
| Token Protocol | Token provides passcode W'; user returns f(r', h(W')) | Passcode theft; replay |
| Static Biometric Protocol | User provides biometric template BT'; returns E(r', D', BT') | Template theft; device authentication |
| Dynamic Biometric Protocol | Host provides random sequence x; user vocalizes/types/writes it | Replay; spoofing |

| Client | Transmission | Host |
|---|---|---|
| $U$, user | $U \rightarrow$ | |
| | $\leftarrow \{r, \text{h}(), \text{f}()\}$ | random number h(), f(), functions |
| $P'$ password $r'$, return of $r$ | $\text{f}(r', \text{h}(P')) \rightarrow$ | |
| | $\leftarrow$ yes/no | if $\text{f}(r', \text{h}(P')) = \text{f}(r, \text{h}(P(U)))$ then yes else no |

(a) Protocol for a password

| Client | Transmission | Host |
|---|---|---|
| $U$, user | $U \rightarrow$ | |
| | $\leftarrow \{r, \text{h}(), \text{f}()\}$ | $r$, random number h(), f(), functions |
| $P' \rightarrow W'$ password to passcode via token $r'$, return of $r$ | $\text{f}(r', \text{h}(W')) \rightarrow$ | |
| | $\leftarrow$ yes/no | if $\text{f}(r', \text{h}(W')) = \text{f}(r, \text{h}(W(U)))$ then yes else no |

(b) Protocol for a token

| Client | Transmission | Host |
|---|---|---|
| $U$, user | $U \rightarrow$ | |
| | $\leftarrow \{r, \text{E}()\}$ | $r$, random number E(), function |
| $B' \rightarrow BT'$ biometric $D'$ biometric device $r'$, return of $r$ | $\text{E}(r', D', BT') \rightarrow$ | $\text{E}^{-1}\text{E}(r', P', BT') = (r', P', BT')$ |
| | $\leftarrow$ yes/no | if $r' = r$ and $D' = D$ and $BT' = BT(U)$ then yes else no |

(c) Protocol for static biometric

| Client | Transmission | Host |
|---|---|---|
| $U$, user | $U \rightarrow$ | |
| | $\leftarrow \{r, x, \text{E}()\}$ | $r$, random number $x$, random sequence challenge E(), function |
| $B', x' \rightarrow BS'(x')$ $r'$, return of $r$ | $\text{E}(r', BS'(x')) \rightarrow$ | $\text{E}^{-1}\text{E}(r', BS'(x')) = (r', BS'(x'))$ extract $B'$ from $BS'(x')$ |
| | $\leftarrow$ yes/no | if $r' = r$ and $x' = x$ and $B' = B(U)$ then yes else no |

(d) Protocol for dynamic biometric

**Figure 3.10 Basic Challenge-Response Protocols for Remote User Authentication**
*Source*: Based on [OGOR03].

# Multi-Factor Authentication (MFA)

**Definition:**
Authentication using two or more different factors from the four means of authentication.

**The Three Factors (Very Important):**

| Factor | Type | Examples |
|---|---|---|
| Something you KNOW | Knowledge | Password, PIN, security question |
| Something you HAVE | Possession | Smart card, phone (OTP), hardware token |
| Something you ARE | Inherence | Fingerprint, retina scan, voice recognition |

**Key Distinction:**

- **Two-Factor Authentication (2FA):** Password + OTP
- **Multi-Factor Authentication (MFA):** Two or more different factors

**Real-World Example:**
**Colonial Pipeline (2021):** Compromised VPN password; no MFA. Result: $4.4 million ransom, fuel shortage. Lesson: MFA is essential.

Table 3.4 Some Potential Attacks, Susceptible Authenticators, and Typical Defenses

| Attacks | Authenticators | Examples | Typical Defenses |
|---|---|---|---|
| **Client attack** | Password | Guessing, exhaustive search | Large entropy; limited attempts |
| | Token | Exhaustive search | Large entropy; limited attempts, theft of object requires presence |
| | Biometric | False match | Large entropy; limited attempts |
| **Host attack** | Password | Plaintext theft, dictionary/exhaustive search | Hashing; large entropy; protection of password database |
| | Token | Passcode theft | Same as password; 1-time passcode |
| | Biometric | Template theft | Capture device authentication; challenge response |
| **Eavesdropping, theft, and copying** | Password | "Shoulder surfing" | User diligence to keep secret; administrator diligence to quickly revoke compromised passwords; multifactor authentication |
| | Token | Theft, counterfeiting hardware | Multifactor authentication; tamper resistant/evident token |
| | Biometric | Copying (spoofing) biometric | Copy detection at capture device and capture device authentication |
| **Replay** | Password | Replay stolen password response | Challenge-response protocol |
| | Token | Replay stolen passcode response | Challenge-response protocol; 1-time passcode |
| | Biometric | Replay stolen biometric template response | Copy detection at capture device and capture device authentication via challenge-response protocol |
| **Trojan horse** | Password, token, biometric | Installation of rogue client or capture device | Authentication of client or capture device within trusted security perimeter |
| **Denial of service** | Password, token, biometric | Lockout by multiple failed authentications | Multifactor with token |

# Security Issues for User Authentication

# Case Study — ATM Systems Security Problems

▶Background:
Redspin, Inc. auditor report on ATM security vulnerability affecting small to mid-size ATM card issuers.

▶Definitions:

- Cardholder: Individual to whom debit card is issued
- Issuer: Institution that issues debit cards (bank, credit union)
- Processor: Organization providing core data processing and EFT services

▶Two Configurations:

- Figure 3.12a: Point-to-point connection to processor — ATM links directly to processor via leased line
- Figure 3.12b: Shared connection to processor — ATM links to issuer network, then to processor

▶Security Problem:

- Upgrade to Windows and TCP/IP introduced vulnerability
- Only PIN encrypted (3DES); rest of message in clear
- Card number, expiration date, account balances, withdrawal amounts exposed

▶Two Principal Vulnerabilities:

- Confidentiality: Card number, expiration date, account balance can be used for online purchases or duplicate cards
- Integrity: No protection against injecting or altering data in transit

▶Recommended Measures:

- Short-term: Segment ATM traffic; network-level encryption between routers
- Long-term: Encrypt all customer-related information; machine-to-machine authentication; challenge-response protocols

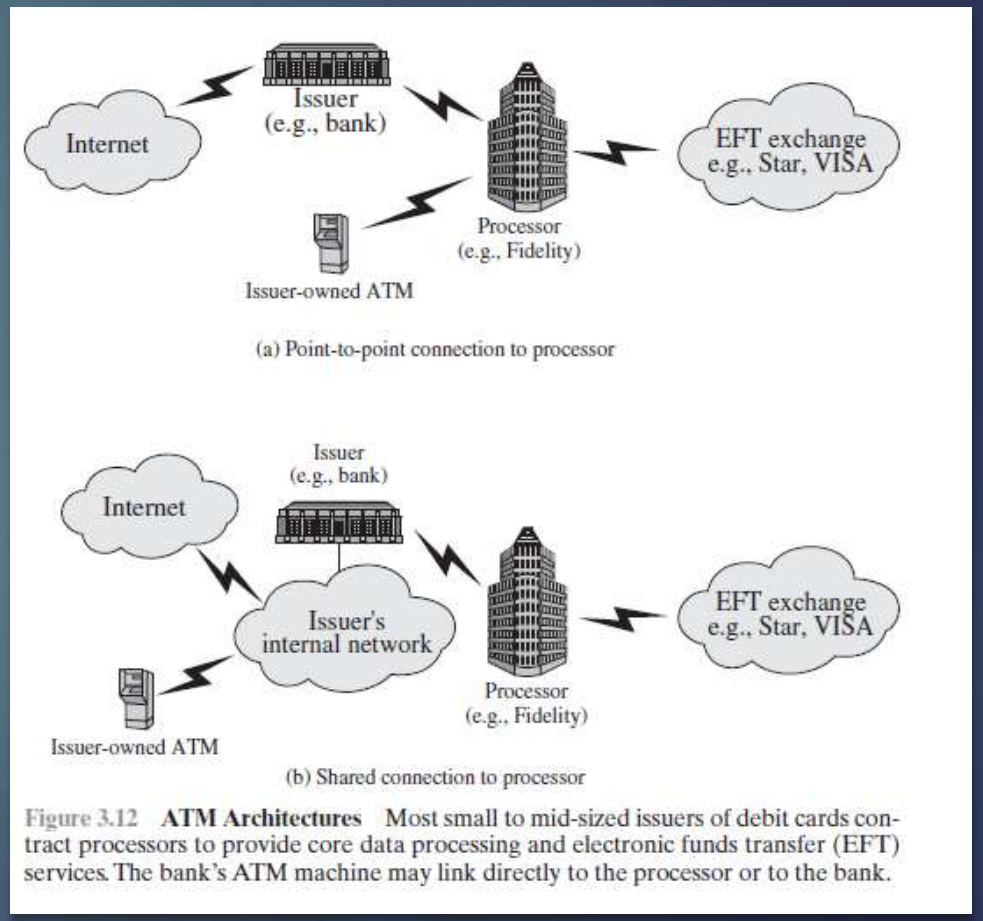



Figure 3.12 **ATM Architectures** Most small to mid-sized issuers of debit cards contract processors to provide core data processing and electronic funds transfer (EFT) services. The bank's ATM machine may link directly to the processor or to the bank.

# Thank you